Project Title	LLM-Powered Analytics Assistant with RAG
Skills take away From This Project	NLP, LLM Integration, RAG, Machine Learning, Deep Learning, Prompt Engineering, SQL, Data Visualisation
Domain	Retail / E-Commerce Analytics

Problem Statement:

Business analysts and operations teams in e-commerce companies spend significant time writing SQL queries and manually reading through thousands of customer reviews to extract actionable insights. This process is slow, error-prone, and inaccessible to non-technical stakeholders who cannot write SQL.

This project aims to build an LLM-Powered Analytics Assistant that enables any business user to query a structured e-commerce database and analyse unstructured customer review text using plain English. The system integrates a Retrieval-Augmented Generation (RAG) pipeline with an NL-to-SQL engine, backed by GPT-3.5/4, to deliver accurate, explainable, and context-grounded answers.

Business Use Cases:

Business analysts and operations teams in e-commerce companies spend significant time writing SQL queries and manually reading through thousands of customer reviews to extract actionable insights. This process is slow, error-prone, and inaccessible to non-technical stakeholders who cannot write SQL.

This project aims to build an LLM-Powered Analytics Assistant that enables any business user to query a structured e-commerce database and analyse unstructured customer review text using plain English. The system integrates a Retrieval-Augmented Generation (RAG) pipeline with an NL-to-SQL engine, backed by GPT-3.5/4, to deliver accurate, explainable, and context-grounded answers.

Approach:

3.1 Data Ingestion & Preparation

- Load the Olist Brazilian E-Commerce dataset (orders, products, reviews, sellers, payments) from Google Drive into a local SQLite database using pandas.
- Chunk all customer review text (order_reviews table) into 200-word segments for embedding.
- Generate dense vector embeddings using the sentence-transformers model all-MiniLM-L6-v2.
- Store embeddings in a FAISS vector index for fast cosine similarity retrieval.

3.2 System Architecture — Three Core Pipelines

- SQL Path (Structured Queries): A schema-aware system prompt is passed to the LLM containing the full table schema. The LLM generates valid SQLite SQL, which is executed via pandas.read_sql(). A second LLM call summarises the resulting dataframe in plain English.
- RAG Path (Unstructured Review Analysis): The user query is embedded, and the top-5 most similar review chunks are retrieved from FAISS. These chunks are injected into the LLM prompt as grounding context. The LLM generates a response with sentiment classification (positive / negative / mixed) and top complaint themes.
- Hybrid Path: For queries requiring both structured and unstructured data, both pipelines are triggered. Results are merged and passed to a final synthesis LLM call.

3.3 Query Router

A lightweight LLM classifier categorises every incoming query into one of three routing classes: SQL, RAG, or HYBRID. This routing decision gates which pipeline is invoked.

3.4 Auto Chart Generation

After every SQL result, an additional LLM prompt determines the best chart type (bar, line, pie) for the returned dataframe. The chart is rendered using Plotly and displayed inline in the Streamlit UI.

3.5 Deployment

The full application is containerised and deployed on Streamlit Community Cloud, with the SQLite database and FAISS index bundled in the repository.

Results:

- A fully functional web application where users can type natural language questions and receive data-grounded answers.
- NL-to-SQL generation with >85% query execution success rate on the Olist schema.
- RAG-based sentiment analysis correctly classifying sentiment and extracting top 3 complaint themes per query.
- Auto-generated Plotly charts for all structured data responses.
- Hybrid query handling demonstrating the integration of both pipelines in a single coherent answer.
- A live Streamlit deployment accessible via public URL.

Project Evaluation metrics:

Section	Marks	Reasoning
Pipeline Correctness	40	SQL + Routing + RAG — core functionality
Output Quality	35	Sentiment + Hallucination + Charts + Completeness
System Performance	10	Latency + Stability
Code, Docs & Deployment	15	Structure + GitHub + Live URL + README

GUVI | HCL
Skill Up. Level Up

Technical Tags:

Large Language Models (LLM) | Retrieval-Augmented Generation (RAG) | NL-to-SQL | Prompt Engineering | Natural Language Processing (NLP) | Sentiment Analysis | FAISS Vector Store | Sentence Transformers | GPT-3.5 / GPT-4 API | PyTorch | pandas | SQLite | Plotly | Streamlit | Python

Data Set:

Dataset Name	Brazilian E-Commerce Public Dataset by Olist
Source	Originally from Kaggle — to be downloaded and hosted on Google Drive (direct Kaggle links not permitted per guidelines)
Size	~100,000 orders, 8 relational tables, ~40,000 review text records
Format	CSV files loaded into SQLite
Access	Learner must download, upload to personal Google Drive, and share the Drive link in their submission

Data Set Explanation:

The Olist dataset is a real Brazilian e-commerce dataset covering 2016–2018. It contains 8 relational tables:

- orders — order lifecycle (status, purchase date, delivery date)
- order_items — product-level line items with price and freight value
- order_reviews — customer-written review text and star ratings (1–5)
- products — product metadata including category
- sellers — seller location and ID
- customers — customer location and ID
- payments — payment type and instalment details
- geolocation — zip code to latitude/longitude mapping

Preprocessing steps required:

- Merge tables using pandas to create a denormalised analytics view for SQL querying.
- Filter and clean review_comment_message (remove nulls, normalise encoding).
- Chunk review text into ~200-word segments with order_id metadata preserved.
- Generate and persist FAISS embeddings at startup to avoid re-computation.

Project Deliverables:

Learners must submit all of the following:

1. GitHub Repository with clean folder structure, README, and requirements.txt
2. Streamlit Application deployed on Streamlit Community Cloud (public URL required)
3. Jupyter Notebook documenting EDA, pipeline development, and evaluation results
4. Google Drive link to dataset (direct Kaggle link not accepted)
5. Project Report (this document) with all sections completed — no missing sections
6. Short demo video (3–5 minutes) walking through the three query paths: SQL, RAG, and Hybrid

Project Guidelines:

Originality

- The problem framing, query routing logic, and hybrid pipeline design must be the learner's own work. Using Olist dataset is permitted but the implementation must not be a direct replication of any public GitHub / Kaggle notebook.
- Feature engineering choices and prompt designs must demonstrate clear differentiation from generic tutorials.

Coding Standards

- All code must be modular — separate files for RAG, SQL, routing, and UI layers (see recommended folder structure below).
- Use environment variables (.env file) for API keys — never hardcode keys in source files.
- Include docstrings for all functions and a top-level README explaining how to run the project locally.

Recommended Folder Structure

```
rag-analytics-assistant/ ├── data/           olist_loader.py ├── rag/
embedder.py, retriever.py ├── sql/            nl_to_sql.py ├── llm/
router.py, synthesizer.py, sentiment.py ├── app.py          Streamlit UI └──
README.md
```

Version Control

- Use Git with meaningful commit messages. Minimum 5 commits expected showing progressive development.
- Do not commit the .env file or API keys. Add to .gitignore.

Deployment

- Deploy on Streamlit Community Cloud (preferred), Render, or AWS.
- The live application URL must be included in the README and this project document.

Timeline: 2 Weeks

Phase	Tasks	Duration
Week 1 — Days 1–2	Dataset download, Drive setup, SQLite ingestion, EDA	2 days
Week 1 — Days 3–4	RAG pipeline: chunking, embedding, FAISS index build	2 days
Week 1 — Days 5–7	NL-to-SQL engine, query router, schema prompt design	3 days
Week 2 — Days 1–2	Hybrid pipeline, sentiment layer, auto chart generation	2 days
Week 2 — Days 3–4	Streamlit UI integration, end-to-end testing, evaluation	2 days
Week 2 — Days 5–6	Deployment on Streamlit Cloud, README, demo video recording	2 days
Week 2 — Day 7	Final review, submission, document completion	1 day

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.

Timing: Monday-Saturday (3:30PM to 4:30PM)

Booking link : Project Doubt Session [DS/AIML]

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.
Note: This form will Open only on Saturday (after 2 PM) and Sunday on Every Week

Timing:
Monday-Saturday (05:30PM to 07:00PM)

Booking link : https://forms.gle/1m2Gsro41fLtZurRA